

LECTURE 14

Synchronization Tools: Mutex Locks, Semaphores, and Monitors

CSC323 – Operating Systems

COMSATS University Islamabad, Lahore Campus

In This Lecture

- Two hardware instructions, `test_and_set()` and `compare_and_swap()`, and the property that lets them succeed where Peterson's solution struggled: hardware-guaranteed atomicity
- Atomic variables, briefly, as a convenient layer built on top of those instructions
- Mutex locks: `acquire()/release()`, spinlocks, and when busy-waiting is genuinely the right engineering choice
- Semaphores: `wait()/signal()`, counting vs. binary semaphores, and enforcing a required order between two processes' events
- Why a correct semaphore implementation must avoid busy-waiting too, by blocking the caller
- Monitors: automatic mutual exclusion plus condition variables, and how a condition variable's `wait()/signal()` differs from a semaphore's

Hardware Support: Atomic Instructions

- The core problem with Peterson's solution is that nothing in it is atomic — every read and write is a separate, interruptible step, which is exactly why reordering can break it.
- Modern CPUs provide instructions that the hardware itself guarantees execute as a single, uninterruptible unit: no other core can observe, or interleave with, any intermediate state partway through.

test_and_set()

```
boolean test_and_set(boolean *target) {  
    boolean rv = *target;  
    *target = true;  
    return rv;  
}  
  
/* reads, unconditionally sets to true, and returns  
   the OLD value — all as one atomic hardware step */
```

A Lock Built From test_and_set()

```
boolean lock = false;

void acquire_lock() {
    while (test_and_set(&lock)) {
        /* busy wait */
    }
}

void release_lock() {
    lock = false;
}
```

compare_and_swap()

```
int compare_and_swap(int *value, int expected, int new_value) {
    int temp = *value;
    if (temp == expected) {
        *value = new_value;
    }
    return temp;
}

int lock = 0; /* 0 = free, 1 = held */
void acquire_lock() {
    while (compare_and_swap(&lock, 0, 1) != 0) { /* busy wait */ }
}
void release_lock() { lock = 0; }
```

Atomic Variables

```
void atomic_inc(int *v) {  
    int old;  
    do {  
        old = *v;  
    } while (compare_and_swap(v, old, old + 1) != old);  
}  
  
/* useful exactly when the only thing being protected  
   is a single number – a reference count, a request counter */
```

Mutex Locks

- A mutex lock (mutual exclusion lock) is the simplest synchronization tool: a lock object with `acquire()` and `release()`, used to bracket a critical section.
- The implementations shown are spinlocks — if `acquire()` cannot get in immediately, it busy-waits, repeatedly re-checking rather than yielding the CPU.

When Busy-Waiting Is the Right Choice

- The critical section is short, so the expected wait is brief.
- The system is a multiprocessor, so some other core is actively running the lock holder and is likely to finish soon.
- The cost of blocking and later waking the process (a full context switch) would exceed the cost of simply spinning for that brief wait.
- When any of these doesn't hold — long critical sections, or a single CPU — a lock that blocks instead of spinning is the better design: a correctly implemented semaphore.

SEMAPHORES

wait() and signal()

```
wait(S) {  
    while (S <= 0) {  
        /* busy-wait, naive version shown for clarity */  
    }  
    S--;  
}  
  
signal(S) {  
    S++;  
}
```

Counting vs. Binary Semaphores

- Counting semaphore — can range over any integer value; used to track how many instances of a resource are currently available (file handles, buffer slots, and so on).
- Binary semaphore — restricted to the values 0 and 1; used purely for mutual exclusion, functionally equivalent to a mutex lock.

Semaphores Enforce Event Ordering

```
semaphore sync = 0;

/* Process A */          /* Process B */
statement_a1;            wait(sync);
signal(sync);             statement_b1;

/* sync starts at 0, so wait(sync) in B cannot return
   until A executes signal(sync) – which only happens
   after statement_a1 has completed */
```

“A Happens Before B”

- A mutex only ever answers one question: is someone else in here right now?
- A semaphore can answer a different, more general question: has a specific event already happened?
- No matter which process the scheduler happens to run first, `statement_a1` is now guaranteed to finish before `statement_b1` begins — a guarantee no plain mutex alone can give.

Blocking Instead of Spinning

```
typedef struct {
    int value;
    struct process *waiting_queue;
} semaphore;

void wait(semaphore *S) {
    S->value--;
    if (S->value < 0) {
        add_to_queue(S->waiting_queue, current_process);
        block(current_process); /* off the CPU, not spinning */
    }
}
```

Blocking Instead of Spinning

```
void signal(semaphore *S) {
    S->value++;
    if (S->value <= 0) {
        struct process *P = remove_from_queue(S->waiting_queue);
        wakeup(P);          /* P returns to the ready queue */
    }
}

/* S->value may go negative - its magnitude counts how many
   processes are currently blocked, waiting for a signal() */
```

The wait()/signal() Bodies Must Still Be Atomic

- These bodies must execute atomically — achieved with test_and_set()/compare_and_swap(), or by briefly disabling interrupts for just these few fixed instructions.
- This revisits Lecture 13's crude interrupt-disabling trick, but safely this time: the disabled window is a handful of instructions with a known, bounded length, not an arbitrary, unbounded user-level critical section.

Three Tools, Three Levels of Abstraction

Mutex Lock

acquire()/release() — pure mutual exclusion, busy-waits by default.

Semaphore

wait()/signal() on an integer — mutual exclusion AND event ordering; blocks instead of spinning.

Monitor

Shared data + procedures bundled together — mutual exclusion is automatic.

Monitors

- A monitor is a higher-level, language-provided construct: it bundles a module's shared data together with the procedures that operate on it.
- The compiler or runtime automatically guarantees that at most one process is ever actively executing inside the monitor at a time — the programmer never writes `acquire()/release()` at all.

A Monitor-Based Bounded Buffer

```
monitor BoundedBuffer {  
    int buffer[N];  
    int count;  
    condition not_full;  
    condition not_empty;  
  
    void insert(int item) {  
        if (count == N) { not_full.wait(); }  
        /* place item into buffer */  
        count++;  
        not_empty.signal();  
    }  
}
```

A Monitor-Based Bounded Buffer

```
int remove() {
    if (count == 0) { not_empty.wait(); }
    /* take an item out of buffer */
    count--;
    not_full.signal();
    return item;
}

/* insert()/remove() callers are automatically the only
   one active inside BoundedBuffer – mutual exclusion is free */
```

Condition Variables vs. Semaphore Operations

A CONDITION VARIABLE HAS NO MEMORY; A SEMAPHORE DOES

Semaphore `signal()` always increments the count, even if no process is waiting. A later `wait()` finds that incremented value and returns immediately — the semaphore “remembers” a signal happened.

Condition variable `signal()` wakes one waiting process only if one happens to be waiting at that exact moment. If no process is waiting, the signal has no effect at all and is simply lost.

A process calling a condition variable's `wait()` must release the monitor's mutual-exclusion lock while it sleeps — otherwise no other process could ever enter the monitor to signal it, and the system would deadlock on itself.

Key Takeaways

- `test_and_set()` and `compare_and_swap()` are hardware-guaranteed atomic instructions — a correct mutual-exclusion lock can be built from either in a few lines.
- A mutex lock is the simplest synchronization tool, busy-waiting (a spinlock) by default — reasonable for short critical sections on a multiprocessor, wasteful otherwise.
- A semaphore's `wait()/signal()` does two distinct jobs: ordinary mutual exclusion and event ordering between processes — a capability a plain mutex does not have.
- A correct semaphore implementation blocks the caller on a waiting queue instead of busy-waiting, which is why semaphores scale to long waits where spinlocks do not.
- A monitor bundles shared data with its operations and provides mutual exclusion automatically; its condition variables let a process wait for application-specific conditions, but `signal()` on a condition variable is lost if no one is waiting.